

Search...

Random Forest for Image Classification Using OpenCV

Last Updated : 05 Nov, 2024

Random Forest is a machine learning algorithm that uses multiple decision trees to achieve precise results in classification and regression tasks. It resembles the process of choosing the best path amidst multiple options. OpenCV, an open-source library for computer vision and machine learning tasks, is used to explore and extract insights from visual data. The goal here is to classify images, particularly focusing on discerning Parkinson's disease through spiral and wave drawings, using Random Forest and OpenCV's capabilities.

What is Random Forest?

[Random Forest](#) is a machine learning algorithm that belongs to the ensemble learning group. It works by constructing a multitude of decision trees during the training phase. The decision of the majority of trees is chosen by the random forest algorithm as the final decision. In the case of regression, it takes the average of the output of different trees, and in the case of classification, it takes the mode of different tree outputs.

How are we going to apply random forest for image classification?

- To apply Random Forest for image classification, we first need to extract features from the images. One common approach is to use pre-trained [convolutional neural networks \(CNNs\)](#) such as [VGG](#) , [ResNet](#) , or [Inception](#) to extract features. These networks are trained on large datasets like ImageNet and have learned to extract meaningful features from images.
- Once we have extracted features from the images, we can use these features as input to the Random Forest algorithm. Each image will be represented by a set of features, and the Random Forest algorithm will learn to classify images based on these features.

- Training a Random Forest for image classification involves splitting the dataset into training and validation sets. The training set is used to train the Random Forest model, while the validation set is used to evaluate its performance. We can tune hyperparameters such as the number of trees in the forest, the maximum depth of the trees, and the number of features to consider at each split using techniques like grid search or random search.
- After training the Random Forest model, we can use it to classify new images by extracting features from the images and passing them to the model for prediction. The model will output a class label for each image, indicating the predicted class of the image based on the features extracted from it.

Implementation: Random Forest for Image Classification Using OpenCV

- The task involves using machine learning techniques, specifically Random Forest, to identify Parkinson's disease through spiral and wave drawings.
- Traditional diagnostic methods struggle with the complexity of these drawings, which vary in style, scale, and quality.
- The goal is to develop a reliable classification system that distinguishes between drawings with and without Parkinson's disease, contributing to early detection and intervention, ultimately improving patient outcomes and quality of life.

Importing the necessary libraries

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import os
import matplotlib.pyplot as plt
from skimage.feature import hog
import random
import cv2
```

Reading the Images

You can download the dataset from [here](#). Use the following command to unzip the file:

```
!unzip /content/drawings.zip -d drawing
```

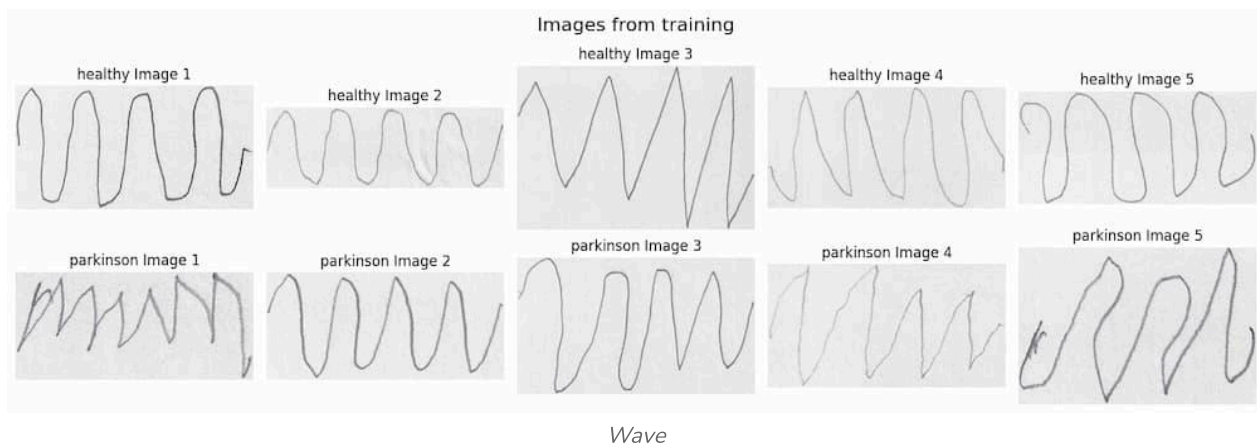
```
def display_images(directory, num_images=5):
    fig, axes = plt.subplots(2, num_images, figsize=(15, 5))
    fig.suptitle(f"Images from {directory.split('/')[1]}"', fontsize=16)

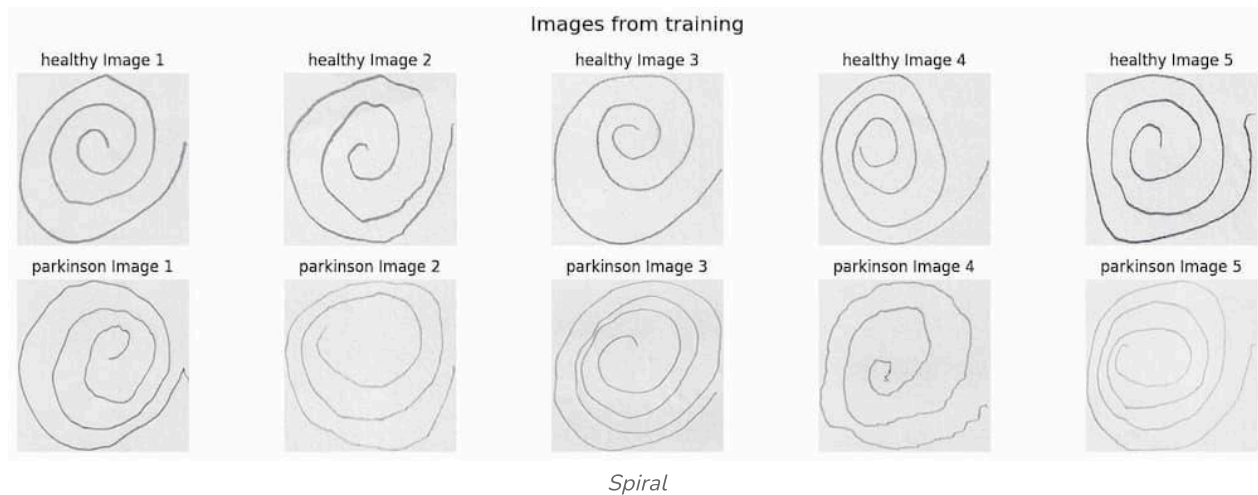
    for i, label in enumerate(os.listdir(directory)):
        label_dir = os.path.join(directory, label)
        image_files = os.listdir(label_dir)
        random.shuffle(image_files)
        for j in range(num_images):
            image_path = os.path.join(label_dir, image_files[j])
            img = cv2.imread(image_path)
            axes[i, j].imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
            axes[i, j].set_title(f"{label} Image {j+1}")
            axes[i, j].axis('off')
    plt.tight_layout()
    plt.show()

# Display training images
display_images('/content/drawing/drawings/spiral/training')
display_images('/content/drawing/drawings/wave/training')

# Display testing images
display_images('/content/drawing/drawings/spiral/testing')
display_images('/content/drawing/drawings/wave/testing')
```

Output:





So, this is the dataset we are working with and will help us to classify based on images if the person has Parkinson's or not.

HOG features are crucial for extracting gradient orientations from images, enabling effective object detection and recognition tasks. They provide a compact representation of an image's structure, reducing computational complexity and preventing overfitting. HOG features are invariant to lighting and color changes, improving generalization ability of models. They are compatible with machine learning algorithms like Support Vector Machines (SVMs) and Random Forests, allowing images to be represented as feature vectors for training and prediction.

Let's understand the code in detail:

hog -is the function that calculates the HOG features based on the following parameters:

- image - the input image for which we need the hog features
- orientations - the number of bins in the histogram
- pixels_per_cell - the size of a cell over which gradient histogram is computed
- cells_per_block - the number of cells in each block
- visualize - whether to return an image of HOG descriptors

shape and structure of objects in an image, making them useful for tasks like object detection and recognition. The parameters **orientations**, **pixels_per_cell**,

and **cells_per_block** control the level of granularity and detail in the computed HOG features.

```
def extract_hog_features(image):  
    # Calculate HOG features  
    hog_features = hog(image, orientations=9, pixels_per_cell=(8, 8),  
        cells_per_block=(2, 2), visualize=False)  
    return hog_features
```

This function is used to load images from a specific directory, resize them, convert them to grayscale and then extracting HOG features from it.

Here we use OpenCV library to do most of the work as its `imread()` function reads images from file, `resize()` function resizes the image to a particular shape and `cvtColor()` changes the image to grayscale using `cv2.COLOR_BGR2GRAY`. All this is done in order to reduce the compute time since colored images and images of their original size have much more values and this is one of the ways to reduce those values thus reducing computation time.

```
def load_and_extract_features(directory):  
    X = []  
    y = []  
    for label in os.listdir(directory):  
        label_dir = os.path.join(directory, label)  
        for filename in os.listdir(label_dir):  
            image_path = os.path.join(label_dir, filename)  
            # Load image using OpenCV  
            img = cv2.imread(image_path)  
            # Resize image to (128, 128)  
            img_resized = cv2.resize(img, (128, 128))  
            # Convert image to grayscale  
            img_gray = cv2.cvtColor(img_resized, cv2.COLOR_BGR2GRAY)  
            # Calculate HOG features  
            hog_features = extract_hog_features(img_gray)  
            X.append(hog_features)  
            y.append(label)  
    return X, y
```

Define Random Forest Classifier

This function is responsible for training a `RandomForestClassifier` with the provided training data as the function parameters.

Inside the function, a Random Forest classifier object (`rf_classifier`) is initialized using the `RandomForestClassifier` class from `scikit-learn`. The classifier is configured with the following parameters:

- `n_estimators`: The number of trees in the forest. Here, it's set to 1000.
- `criterion`: The function to measure the quality of a split. 'gini' is used here, which refers to the Gini impurity.
- `max_depth`: The maximum depth of the trees. Here, it's set to 5.

The fit method then trains the classifier and finally the trained model is returned.

```
# Define a function to train a Random Forest classifier  
def train_random_forest(X_train, y_train):  
    rf_classifier = RandomForestClassifier(n_estimators=1000, criterion='gini',  
max_depth=5)  
    rf_classifier.fit(X_train, y_train)  
    return rf_classifier
```

After we are done with creating a model we load the images and split them into training and testing data sets and create two different models for spiral drawings and wave drawings respectively.

Model Training

```
# Load and extract features from training data  
spiral_train_X, spiral_train_y =  
load_and_extract_features('/content/drawing/drawings/spiral/training')  
wave_train_X, wave_train_y =  
load_and_extract_features('/content/drawing/drawings/wave/training')  
  
# Train Random Forest classifiers  
spiral_rf_classifier = train_random_forest(spiral_train_X, spiral_train_y)  
wave_rf_classifier = train_random_forest(wave_train_X, wave_train_y)  
  
# Load and extract features from testing data  
spiral_test_X, spiral_test_y =  
load_and_extract_features('/content/drawing/drawings/spiral/testing')  
wave_test_X, wave_test_y =  
load_and_extract_features('/content/drawing/drawings/wave/testing')
```

Model Evaluation

```
spiral_predictions = spiral_rf_classifier.predict(spiral_test_X)  
wave_predictions = wave_rf_classifier.predict(wave_test_X)  
  
spiral_accuracy = accuracy_score(spiral_test_y, spiral_predictions)  
wave_accuracy = accuracy_score(wave_test_y, wave_predictions)
```

```
print(&quot;Spiral Classification Accuracy:&quot;, spiral_accuracy)
print(&quot;Wave Classification Accuracy:&quot;, wave_accuracy)
```

Output:

Spiral Classification Accuracy: 0.7666666666666667

Wave Classification Accuracy: 0.7333333333333333

Advantages of Using Random Forest

- **High Accuracy:** Random Forest is a classification method that uses multiple decision trees to achieve high accuracy, reducing overfitting and generalizing well to unseen data.
- **Robustness to Overfitting:** Random Forest reduces overfitting by aggregating predictions from multiple decision trees trained on random data subsets.
- **Versatility:** Random Forest is a versatile algorithm that can perform both classification and regression tasks, making it suitable for a wide range of applications.
- **Feature Importance:** Random Forest is a tool that aids in identifying the most influential features in a dataset, aiding in feature selection and interpretation of results.
- **Efficiency:** Despite its ensemble nature, Random Forest is computationally efficient, capable of handling large datasets with high dimensionality.
- **Resistance to noise:** Random Forest is a robust method that aggregates predictions from multiple trees, reducing the impact of individual noisy data points.
- **Interpretability:** Random Forest, an ensemble method, offers valuable insights into decision-making through feature importance metrics and visualization techniques, enhancing model interpretation and comprehension.

Disadvantages of Using Random Forest

- **Computational Complexity:** Random Forest can be computationally intensive, especially when dealing with a large number of trees and high-dimensional datasets.

- **Memory Consumption:** Random Forest requires storing multiple decision trees in memory, which can lead to high memory consumption, especially when dealing with large forests or datasets with many features.
- **Difficulty with Imbalanced Datasets:** Random Forest may struggle to handle imbalanced datasets, where one class significantly outweighs the others.
- **Black Box Nature:** Despite efforts to interpret feature importance, Random Forest remains a black box model, making it challenging to understand the underlying relationships between features and predictions.
- **Bias Towards Features with Many Categories:** Random Forest tends to favor features with many categories or levels, potentially inflating their importance in the model. This bias can lead to suboptimal predictions, especially if these features are not genuinely informative.

[Comment](#)[More info](#)[Advertise with us](#)

Next Article

[Quantiles in Machine Learning](#)

Similar Reads

Feature extraction and image classification using OpenCV

This article is your ultimate guide to becoming a pro at image feature extraction and classification using OpenCV and Python. We'll kick things off with an overview of how OpenCV plays a role in feature extraction...

11 min read

Python | Image Classification using Keras

Image classification is a method to classify way images into their respective category classes using some methods like : Training a small network from scratchFine-tuning the top layers of the model using VGG16...

4 min read

Random Forest Classifier using Scikit-learn

Random Forest is a method that combines the predictions of multiple decision trees to produce a more accurate and stable result. It can be used for both classification and regression tasks. In classification tasks,...

5 min read

Image Classification using CNN

The article is about creating an Image classifier for identifying cat-vs-dogs using TFLearn in Python. Machine Learning is now one of the hottest topics around the world. Well, it can even be said of the new electricity in...

7 min read

Image Classification using Google's Teachable Machine

Machine learning is a scientific field that allows computers to learn without being programmed directly. When many learners, students, engineers, and data scientists use machine learning to create diverse projects and...

2 min read

Multiclass image classification using Transfer learning

Image classification is one of the supervised machine learning problems which aims to categorize the images of a dataset into their respective categories or labels. Classification of images of various dog breeds is a class...

9 min read

Color Identification in Images using Python - OpenCV

An open-source library in Python, OpenCV is basically used for image and video processing. Not only supported by any system, such as Windows, Linux, Mac, etc. but also it can be run in any programming...

3 min read

Image Classification Using PyTorch Lightning

Image classification is one of the most common tasks in computer vision and involves assigning a label to an input image from a predefined set of categories. While PyTorch is a powerful deep learning framework,...

5 min read

Dog Breed Classification using Transfer Learning

In this tutorial, we will demonstrate how to build a dog breed classifier using transfer learning. This method allows us to use a pre-trained deep learning model and fine-tune it to classify images of different dog breed...

9 min read

Adding borders to the images using Python - OpenCV

Image processing is an interesting field in today's era of Artificial Intelligence and Machine Learning. We can see the applications of image processing in our day-to-day life, like whenever we apply filter over any image...

1 min read

A-143, 7th Floor, Sovereign Corporate
Tower, Sector- 136, Noida, Uttar Pradesh
(201305)

Registered Address:

K 061, Tower K, Gulshan Vivante
Apartment, Sector 137, Noida, Gautam
Buddh Nagar, Uttar Pradesh, 201305



Advertise with us

Company

About Us
Legal
Privacy Policy
In Media
Contact Us
Advertise with us
GFG Corporate Solution
Placement Training Program

Languages

Python
Java
C++
PHP
GoLang
SQL
R Language
Android Tutorial
Tutorials Archive

DSA

Data Structures
Algorithms
DSA for Beginners
Basic DSA Problems
DSA Roadmap
Top 100 DSA Interview Problems
DSA Roadmap by Sandeep Jain
All Cheat Sheets

Data Science & ML

Data Science With Python
Data Science For Beginner
Machine Learning
ML Maths
Data Visualisation
Pandas
NumPy
NLP
Deep Learning

Web Technologies

HTML
CSS
JavaScript
TypeScript
ReactJS
NextJS
Bootstrap
Web Design

Python Tutorial

Python Programming Examples
Python Projects
Python Tkinter
Python Web Scraping
OpenCV Tutorial
Python Interview Question
Django

Computer Science

- Operating Systems
- Computer Network
- Database Management System
- Software Engineering
- Digital Logic Design
- Engineering Maths
- Software Development
- Software Testing

System Design

- High Level Design
- Low Level Design
- UML Diagrams
- Interview Guide
- Design Patterns
- OOAD
- System Design Bootcamp
- Interview Questions

School Subjects

- Mathematics
- Physics
- Chemistry
- Biology
- Social Science
- English Grammar
- Commerce
- World GK

DevOps

- Git
- Linux
- AWS
- Docker
- Kubernetes
- Azure
- GCP
- DevOps Roadmap

Interview Preparation

- Competitive Programming
- Top DS or Algo for CP
- Company-Wise Recruitment Process
- Company-Wise Preparation
- Aptitude Preparation
- Puzzles

GeeksforGeeks Videos

- DSA
- Python
- Java
- C++
- Web Development
- Data Science
- CS Subjects

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved